

A Review of Andrew Ng's Machine Learning Specialization

Mohammad Javad Akbari, 2025 (AI Written)

This document provides a comprehensive summary of the three courses in Andrew Ng's Machine Learning Specialization on Coursera. It covers the core concepts, mathematical foundations, and practical code implementations for each major topic, serving as a detailed reference for later review.

Course 1: Supervised Machine Learning: Regression & Classification

This foundational course establishes the core principles of supervised learning, where models learn from data that has been manually labeled with the correct outcomes.

Topic 1: Linear Regression

Detailed Explanation:

Linear regression is the "hello world" of machine learning. It's used for regression tasks, where the goal is to predict a continuous numerical value (e.g., the price of a house). The model assumes a linear relationship between the input features (X) and the output target (y). The hypothesis function is a straight line:

$$f_{w,b}(x) = wx + b$$

- x is the input feature.
- w is the **weight**, controlling the slope of the line.
- b is the **bias**, controlling the y-intercept.

To find the "best" line, we use a cost function, typically the Mean Squared Error (MSE), which measures the average squared difference between the predicted values and the actual values. The goal is to find the values of w and b that minimize this cost.

$$J(w,b) = (1/2m) \sum_{i=1}^m (f(x(i)) - y(i))^2$$

This minimization is achieved using Gradient Descent, an iterative optimization algorithm that adjusts w and b in the direction that most steeply reduces the cost.

Code Example (Scikit-learn):

```
1. import numpy as np
2. from sklearn.model_selection import train_test_split
3. from sklearn.linear_model import LinearRegression
4. from sklearn.metrics import mean_squared_error
5.
6. # 1. Create sample data (e.g., years of experience vs. salary)
7. X = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(-1, 1)
8. y = np.array([50, 60, 65, 75, 80, 90, 100, 115])
9.
```

```

10. # 2. Split data for training and testing
11. X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
12.
13. # 3. Create and train the model
14. model = LinearRegression()
15. model.fit(X_train, y_train)
16.
17. # 4. Make predictions and evaluate
18. predictions = model.predict(X_test)
19. print(f"Linear Regression MSE: {mean_squared_error(y_test, predictions):.2f}")
20. print(f"Learned weight (w): {model.coef_[0]:.2f}, Learned bias (b): {model.intercept_: .2f}")
21.

```

Topic 2: Logistic Regression

Detailed Explanation:

Despite its name, logistic regression is used for classification tasks, where the goal is to predict a discrete category (e.g., "spam" or "not spam"). It adapts linear regression by feeding the output of the linear equation ($z=wx+b$) into the sigmoid function.

$$g(z) = 1 / (1 + e^{-z})$$

The sigmoid function squashes any real number into a range between 0 and 1, which can be interpreted as a probability. For example, if the output is 0.8, the model is 80% confident that the sample belongs to the positive class. A decision boundary (typically at a probability of 0.5) is used to make the final classification.

The cost function used is **Binary Cross-Entropy** (or Log Loss), which penalizes the model heavily for being confidently wrong.

Code Example (Scikit-learn):

```

1. from sklearn.linear_model import LogisticRegression
2. from sklearn.datasets import make_classification
3. from sklearn.metrics import accuracy_score
4.
5. # 1. Create sample classification data
6. X, y = make_classification(n_samples=100, n_features=2, n_redundant=0, n_informative=2,
7. random_state=42)
8. X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
9.
10. # 2. Create and train the model
11. model = LogisticRegression()
12. model.fit(X_train, y_train)
13.
14. # 3. Make predictions and evaluate
15. predictions = model.predict(X_test)
16. print(f"\nLogistic Regression Accuracy: {accuracy_score(y_test, predictions)*100:.2f}%")

```

Course 2: Advanced Learning Algorithms

This course introduces more complex, non-linear models and the practical methodologies for applying machine learning effectively.

Topic 1: Neural Networks

Detailed Explanation:

Neural Networks are powerful models that can learn extremely complex patterns. They are composed of layers of interconnected neurons.

- **Architecture:** They have an **input layer** (for the features), one or more **hidden layers** (where the computation happens), and an **output layer** (for the prediction).
- **Activation Functions:** Each neuron in the hidden and output layers applies an activation function, such as **ReLU (Rectified Linear Unit)** or **Sigmoid**, to its input. This is what allows the network to learn non-linear relationships.
- **Forward Propagation:** The process where input data is passed through the network, layer by layer, to produce an output prediction.
- **Backpropagation:** The algorithm that trains the network. It calculates the error of the prediction and propagates this error backward through the network, using Gradient Descent to update the weights of each neuron to minimize the error.

Code Example (TensorFlow/Keras):

```
1. import tensorflow as tf
2. # (Using the same classification data from the previous example)
3.
4. # 1. Define the Neural Network architecture
5. model = tf.keras.Sequential([
6.     # Input layer and first hidden layer
7.     tf.keras.layers.Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
8.     # Second hidden layer
9.     tf.keras.layers.Dense(16, activation='relu'),
10.    # Output layer for binary classification
11.    tf.keras.layers.Dense(1, activation='sigmoid')
12. ])
13.
14. # 2. Compile the model
15. model.compile(optimizer='adam',
16.               loss='binary_crossentropy',
17.               metrics=['accuracy'])
18.
19. # 3. Train the model
20. model.fit(X_train, y_train, epochs=50, verbose=0)
21.
22. # 4. Evaluate the model
23. loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
24. print(f"\nNeural Network Accuracy: {accuracy*100:.2f}%")
25.
```

Topic 2: Decision Trees and Ensemble Methods

Detailed Explanation:

- **Decision Trees:** These are intuitive models that create a flowchart of questions to arrive at a prediction. At each node, the tree splits the data based on the feature that provides the most **Information Gain** (i.e., creates the purest subgroups). While easy to understand, single trees are highly prone to **overfitting**.
- **Ensemble Methods:** These methods combine many "weak" decision trees to create a single, robust model, which significantly reduces overfitting.
 - **Random Forest:** Builds hundreds of trees in parallel. Each tree is trained on a random sample of the data and can only consider a random subset of features at each split. The final prediction is a majority vote of all the trees.
 - **Gradient Boosting (e.g., XGBoost):** Builds trees sequentially. Each new tree is specifically trained to correct the errors made by the previous trees. This often leads to higher performance but can be more sensitive to tuning.

Code Example (Scikit-learn):

```
1. from sklearn.ensemble import RandomForestClassifier
2.
3. # (Using the same classification data from the previous example)
4.
5. # 1. Create and train a Random Forest model
6. model = RandomForestClassifier(n_estimators=100, random_state=42)
7. model.fit(X_train, y_train)
8.
9. # 2. Make predictions and evaluate
10. predictions = model.predict(X_test)
11. print(f"\nRandom Forest Accuracy: {accuracy_score(y_test, predictions)*100:.2f}%")
12.
```

Course 3: Unsupervised Learning, Recommenders, & Reinforcement Learning

This course ventures into machine learning problems where the data is unlabeled or the learning process is interactive.

Topic 1: Unsupervised Learning

Detailed Explanation:

- **Clustering (K-Means):** The goal is to partition data into a pre-specified number (K) of clusters. The algorithm iteratively assigns data points to the nearest cluster **centroid** and then updates the centroid's position to the mean of its assigned points. It's used for tasks like customer segmentation.

- **Dimensionality Reduction (PCA): Principal Component Analysis** is a technique used to reduce the number of features in a dataset by transforming them into a new, smaller set of uncorrelated features called **principal components**. These components capture the maximum possible variance in the data. It is widely used for data visualization and compression.

Code Example (Scikit-learn):

```
1. from sklearn.cluster import KMeans
2. from sklearn.decomposition import PCA
3. from sklearn.datasets import load_digits
4.
5. # 1. Load high-dimensional data (64 features)
6. digits = load_digits()
7. X = digits.data
8.
9. # 2. Reduce dimensions from 64 to 2 using PCA
10. pca = PCA(n_components=2, random_state=42)
11. X_reduced = pca.fit_transform(X)
12.
13. # 3. Cluster the reduced data using K-Means
14. kmeans = KMeans(n_clusters=10, random_state=42, n_init='auto')
15. clusters = kmeans.fit_predict(X_reduced)
16. print("\nUnsupervised Learning: PCA and K-Means models created and applied.")
17. # A plot of X_reduced colored by 'clusters' would show the groupings.
18.
```

Topic 2: Recommender Systems

Detailed Explanation:

- **Collaborative Filtering:** This method leverages the "wisdom of the crowd." It recommends items to a user based on the preferences of similar users. Modern approaches use **Matrix Factorization** to learn **latent feature vectors (embeddings)** for every user and every item. The dot product of a user's vector and an item's vector predicts the rating, and the model is trained to make these predictions match known ratings.
- **Content-Based Filtering:** This method recommends items based on their intrinsic properties. It builds a profile of a user's tastes based on the features of items they've liked (e.g., genre, actors, keywords). It then recommends new items with similar features.

Code Example (Content-Based Filtering):

```
1. from sklearn.feature_extraction.text import TfidfVectorizer
2. from sklearn.metrics.pairwise import linear_kernel
3.
4. # 1. Sample movie descriptions
5. descriptions = [
6.     'A brave hero goes on a thrilling space adventure',
7.     'A hilarious comedy about a family vacation',
8.     'A lone hero fights aliens on a distant spaceship'
9. ]
10.
```

```
11. # 2. Convert text to numerical vectors
12. tfidf = TfidfVectorizer()
13. tfidf_matrix = tfidf.fit_transform(descriptions)
14.
15. # 3. Compute the similarity between all movies
16. cosine_sim = linear_kernel(tfidf_matrix, tfidf_matrix)
17. print("\nMovie Similarity Matrix:\n", np.round(cosine_sim, 2))
18. # The output shows that movie 0 and 2 are highly similar.
19.
```

Topic 3: Reinforcement Learning (RL)

Detailed Explanation:

RL is a paradigm for training an agent to make optimal decisions through trial and error. The agent interacts with an environment in a continuous loop:

1. The agent observes the current **state** (S) of the environment.
2. It takes an **action** (A).
3. The environment provides a numerical reward (R) and transitions to a new state.

The agent's goal is to learn a policy (a strategy) that maximizes its total cumulative reward over time. A central concept is the Q-value function, $Q(s,a)$, which estimates the long-term value of taking a specific action in a specific state. Algorithms like Q-Learning and Deep Q-Networks (DQN) are designed to learn this function, often by balancing exploration (trying new actions) and exploitation (using known good actions).